SynthWorks

VHDL Training Experts

AlertLogPkg and ReportPkg Quick Ref

1. Simple Mode: Basic Errors and Messages

Simple mode tracks alerts, logs, and affirmations with a single error and affirmation counters. This is sufficient for basic test cases.

1.1 Package References

```
library osvvm ;
context osvvm.OsvvmContext ;
```

1.2 SetTestName

Sets the test name. Printed by ReportAlerts.

```
SetTestName("Tb_SendGet_1") ;
```

1.3 Log Levels

Logs have the levels ALWAYS, DEBUG, FINAL, INFO, and PASSED. Logs are disabled by default – except for ALWAYS. SetLogEnable enables logs.

1.4 Logs

Logs are for conditional printing. Logs allow printing of lots of messages when debugging and very few when running regressions. Log prints when the LogLevel is enabled or the optional enable (last parameter) is TRUE.

```
Log("Uart Parity Received", INFO) ;
--                                          Enable
Log("Received read responset", DEBUG, TRUE) ;
```

1.5 Log Enable / Disable

Enable all logs of a particular log level:

```
--              Level,  Enable
SetLogEnable(INFO,  TRUE) ;
```

1.6 Alert Levels

Alerts have levels FAILURE, ERROR, and WARNING.

1.7 Alerts

Alerts are for checking for proper behavior – generate an error when a response is unexpected. AlertLevel ERROR is the default. Alerts are enabled by default (and rarely disabled).

```
Alert("Uart Parity") ;
AlertIf(Break='1', "Uart Break", ERROR) ;
AlertIfNot(ReadValid, "Read", FAILURE);
AlertIfDiff("FileA.txt","FileB.txt");
```

1.8 Alert Enable / Disable

Alerts can be enabled or disabled in a similar fashion to logs, however, this is rarely done. When it is done, errors are still tracked and reported as disabled alerts.

1.9 Affirmations

Affirmations are for checking the received (actual) value against an expected value – the heart of self-checking tests. If the check matches, a log PASSED is produced, otherwise, an alert ERROR is produced.

```
AffirmIf(Data = Expected, "Data: " & to_string(Data),
   " /=  Expected: " & to_string(Expected)) ;
AffirmIfNot(ReadValid, "Reading Test.txt") ;
AffirmIfEqual(ReceivedData, ExpectedData, "Data") ;
AffirmIfFilesMatch("Uart1.txt", "validated/Uart1.txt") ;
```

1.10 EndOfTestReports

Write out text report and YAML based alert reports.

```
EndOfTestReports ;
```

See AlertLog_user_guide.pdf for additional parameters.

2. Hierarchy Mode

Hierarchy mode tracks alerts, logs, and affirmations from different sources separately. An AlertLogID is used to identify each source.

All alerts, logs, and affirmations support both simple and hierarchy mode. When used the AlertLogID is always specified as the first parameter of the call.

Hierarchy mode requires the following additional steps or modified steps.

2.1 NewID: Create Hierarchy

Create an AlertLogId and associate it with the name.

```
Signal ModelID, ProtocolID : AlertLogIDType ;
. . .
ModelID <= NewID("Uart_1") ;
wait for 0 ns ;
--                              Name,       Parent AlertLogID
ProtocolID <= NewID("Protocol",  ModelID);
```

If an ID is not specified, its parent is the top ID, ALERTLOG_BASE_ID.

2.2 PathTail

PathTail is used in conjunction with PATH_NAME to determine an instance name for an entity.

```
ModelID  <=  NewID(PathTail(UART'PATH_NAME));
```

2.3 IDs and Alerts, Logs, Affirmations

When using an AlertLogID, it is the first parameter of the procedure being called.

```
AlertIf(ModelID, Break='1', "Uart Break", ERROR) ;
Log(ModelID, "Uart Parity Received", INFO) ;
AffirmIfEqual(ModelID, ReceivedData, ExpectedData) ;
```

2.4 Log Enable / Disable

SetLogEnable will also enable the children of an AlertLogID if the DescendHierarchy is TRUE.

```
--              AlertLogID, Level,  Enable, DescendHierarchy
SetLogEnable(ModelID, INFO, TRUE,  TRUE) ;
```

2.5 Read Log Enables

Log enables can also be read from a file using:

```
ReadLogEnables("InitEnables.txt") ;
```

http://www.SynthWorks.com jim@SynthWorks.com

3. Multiline Messages

By default, the message is printed on the same line as the alert or log – independent of how long the line is.

If a line starts with either LF or '<', the message will be printed on the next line with a short indent. If the message exceeds OSVVM_LINE_WRAP characters, additional lines will be produced at the wrap point and will be indented to match the preceding line.

If a line starts with '>', the message will be printed on the same line as the Alert or Log. If the message exceeds the OSVVM_LINE_WRAP characters, additional lines will be produced at the wrap point and will be indented to match the preceding line.

Otherwise if the line contains an LF character, the line will wrap at the LF.

4. LogHeader

LogHeader is a log message plus additional blank lines or repeated characters to make the line more visible. Each character in the prefix and suffix strings indicates a line to print. If the character is a space, a blank line is printed, otherwise, a line of that character is printed.

```
LogHeader("UART1 Test", " *", "* ") ;
```

Prints as:

```
%%
%% ************************************************
%%      50.0 ns Log ALWAYS      UART1 Test
%% ************************************************
%%
```

Calling LogHeader without Prefix and Suffix is equivalent to the following:

```
LogHeader("UART1 Test", " ", "*") ;
```

5. ClearAlerts

Reset alert counts to 0. Used in an environment where a VC is erroneously signaling an error at the start of the test.

```
ClearAlerts ;
```

6. Options for Reports

Options for AlertLogPkg are determined by deferred constants in OsvvmSettingsPkg. See OSVVM Settings User Guide for details. The defaults are intended to help you get started.

7. Controlling IDs from Other Locations

Alerts and Logs may need to be enabled (or disabled) from a test case which is in a different entity than the VC that defined it. To do this, the test case needs to be able to look up the AlertLogID for the VC. No problem. AlertLogIDs can be looked up using GetID and the same string used in its definition as done in the following code.

```
signal TBID, AxiManagerID, UartTxID, UartRxID :
AlertLogIDType ;
. . .
ControlProc : process
begin
  TBID        <= NewID("TB") ;
  wait for 0 ns ;  -- Allow the VC to create the ID
  AxiManagerID <= GetID("AxiManager_1") ;
  UartTxID     <= GetID("UartTx_1") ;
  UartRxID     <= GetID("UartRx_1") ;
```

The following code turns on PASSED for all IDs and INFO for the UartTxID. This capability facilitates finding infrequent messages (such as from a slow UART) hidden among messages from faster interfaces.

```
  . . .
SetLogEnable(PASSED, TRUE) ;
SetLogEnable(UartTxID, INFO, TRUE) ;
```

8. File IO / Transcript Files

Alert, Log, and Affirmations all print using OSVVM's transcript capability, giving the test case full control of where the output goes (std.textio.OUTPUT, TranscriptFile, or both).

9. Alert Stop Counts

SetAlertStopCount stops a simulation after a specified number of alerts have occurred. This debug to start sooner.

Use SetAlertStopCount without an AlertLogID to set the stop count for accumulated alerts across the entire test case.

```
--                      Level,        Count
SetAlertStopCount(ERROR,     20) ;
```

Use SetAlertStopCount with an AlertLogID to control the stop count for a particular verification component.

```
--                  AlertLogID,  Level,        Count
SetAlertStopCount(UartID, ERROR,     20) ;
```

10. Alert Print Counts

AlertPrintCount stops printing an alert after a specified number of alerts have occurred. This allows acknowledging an issue, but stops its output from overwhelming the simulation output.

SetAlertPrintCount can be used with or without an ID.

```
--                      Level,        Count
SetAlertPrintCount(ERROR,     20) ;

--                  AlertLogID,    Level,        Count
SetAlertPrintCount(UartTxID, ERROR,     20) ;
```


SynthWorks Design Inc.
VHDL Intro, RTL, and Verification Training
11898 SW 128th Ave. Tigard OR 97223 +1-503-590-4787
http://www.SynthWorks.com jim@synthworks.com